ACADEMIC COMPILER & ENGINE PROJECT

Shader Program Compiler

2D/3D Graphics & Game DSL Engine

A Declarative High-Level Domain-Specific Language (DSL) & C++ Transpiler Compiling to Native Hardware-Accelerated OpenGL 2.1 + SDL2 Binaries

C++17 Transpiler | Hardware OpenGL 2.1 | Real-time GLSL Shaders | 2D Pong with AI Physics | 3D First-Person POV World

Student Author: Eldho | Semester 3 Mini Project | Repository: github.com/eldhoGI2/shader-compiler

PROJECT OVERVIEW

Executive Summary: The Bridge from High-Level Script to GPU Native Code

1. Declarative DSL (.my)

• High-level declarative syntax that replaces hundreds of lines of OpenGL boilerplate.
• Expressive commands: 'window 800 600', 'bg #060614', 'shader grid', 'pong', 'mode_3d'.
• 35+ named color dictionary (cyan, crimson, gold, neon_green) + hex codes (#RRGGBB).
• Built-in 2D shapes, outlines, transformations, and multi-color linear gradients.

2. Automated Transpiler

• Full source-to-source compiler architecture built with modern C++ (Lexer, Parser, AST, CodeGen).
• Translates .my domain language into high-efficiency intermediate C code (generated_game.c).
• Automatically invokes GCC and links hardware libraries (-lSDL2 -lGL -lGLU -lm).
• Produces an optimized, standalone 60 FPS desktop binary in one seamless step: ./mycompiler script.my.

3. GPU Hardware Engine

• OpenGL 2.1 + SDL2 backend engine with hardware-accelerated vertex & fragment pipeline.
• 8+ real-time GLSL fragment shaders (Plasma, Cyberpunk Grid, Waves, CRT, Starfield).
• Interactive 2D Pong with ball reflection physics, AI paddle tracking, and live retro scoreboard.
• Navigable 3D first-person POV world with WASD movement, mouse camera, and wall collision.

CHALLENGE & MOTIVATION

The Problem: The Extreme Verbosity of Raw C & OpenGL

Why Low-Level C Graphics Frustrates Developers

Massive Boilerplate Overhead

To draw a single circle or rectangle, a developer must write 50+ lines of window subsystem initialization, pixel formats, and GL context setup.

Manual OpenGL State Machine

Developers must manually manage matrix stacks (GL_PROJECTION, GL_MODELVIEW), orthogonal projections, vertex arrays, and double-buffering.

Complex GLSL Shader Lifecycle

Compiling shaders requires manual shader object creation, source loading, error log querying, program attachment, uniform location caching, and linking.

High Cognitive Barrier

Students and designers spending 90% of their time fighting syntax and API quirks instead of focusing on graphics logic, animations, or gameplay.

Actual Raw C + OpenGL 2.1 Code (50+ Lines Required)

The Problem: Raw C + OpenGL 2.1 Boilerplate (50+ Lines)

```c
// Creating a single window and drawing a primitive in raw C / OpenGL:
#include <SDL2/SDL.h>
#include <SDL2/SDL_opengl.h>

SDL_Init(SDL_INIT_VIDEO);
SDL_GL_SetAttribute(SDL_GL_CONTEXT_MAJOR_VERSION, 2);
SDL_GL_SetAttribute(SDL_GL_CONTEXT_MINOR_VERSION, 1);
SDL_Window* win = SDL_CreateWindow("App", 0, 0, 800, 600, SDL_WINDOW_OPENGL);
SDL_GLContext ctx = SDL_GL_CreateContext(win);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(0, 800, 600, 0, -1, 1);
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();

// Event loop & manual rendering...
while (running) {
    while (SDL_PollEvent(&ev)) { /* Handle quit, keys, mouse */ }
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_POLYGON);
    /* 360 circle trigonometric vertices... */
    glEnd();
    SDL_GL_SwapWindow(win);
}
```

THE DSL SOLUTION

How We Made It Easier: Declarative Syntax & One Command, One Intent

The Declarative Language Philosophy

One Intent = One Readable Line

Replacing 50 lines of C boilerplate with 'window 800 600', 'bg #060614', 'fill_circle 400 300 80 red'.

Zero Boilerplate Programming

The programmer declares WHAT should be on screen, while the compiler and runtime handle HOW the GPU renders it.

Color & Math Abstraction

Built-in 35+ named color dictionary (gold, cyan, lime, coral), hex support (#00d2ff), and automatic degree-to-radian conversion.

Instant Game & 3D Modes

Activating full interactive physics and 3D camera with single declarative keywords: 'pong' and 'mode_3d'.

The DSL Equivalent: 4 Clean, Readable Lines

The Solution: Declarative DSL (.my) – Clean, readable & fast

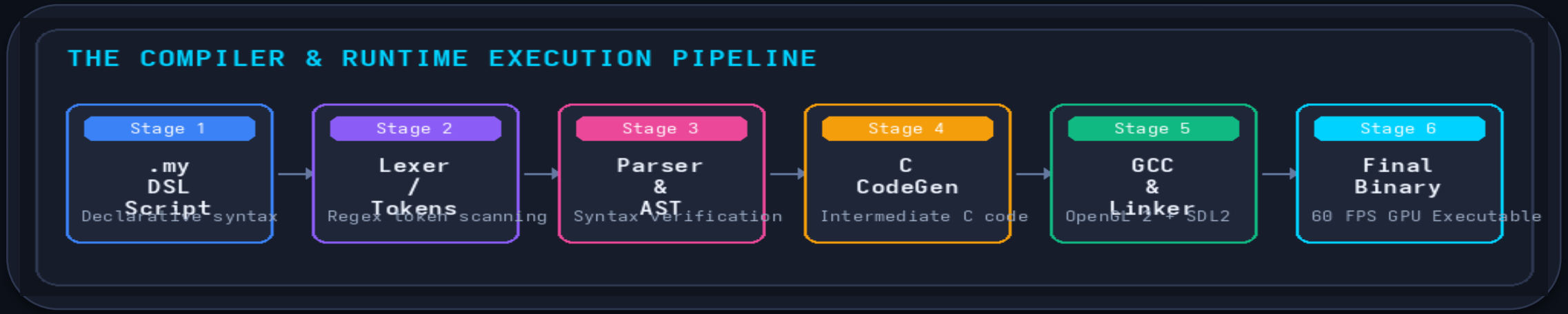
```
# Declarative DSL: Same program in 4 clean, readable lines
window 800 600
title "Graphics Application"
bg 10 12 25
fill_circle 400 300 80 red
```

Code Reduction & Efficiency Impact

- Lines of Code: Reduced from 55 lines to 4 lines (92.7% reduction)
- Compilation Time: < 0.6 seconds from .my script to running native binary
- Error Prevention: Zero dangling pointers, zero GL state leaks, zero context bugs.

COMPILER PIPELINE

System Architecture: End-to-End Transpiler & Execution Pipeline



1. Frontend (Lexer & AST)

- Reads .my source file and strips comments.
- Tokenizes into classified tokens (TOKEN_WINDOW, TOKEN_CUBE, TOKEN_PONG).
- Validates parameters and builds structured AST.

2. Transpiler CodeGen

- Emits ANSI C code into generated_game.c.
- Injects #include "graphics_runtime.h".
- Packages draw calls into high-speed compiled OpenGL display lists (glNewList).

3. GCC Linking & Binary

- Invokes GCC automatically via system().
- Flags: -lSDL2 -lGL -lGLU -lm.
- Outputs final_game: a 60 FPS standalone native desktop executable with GPU support.

FILE-BY-FILE GUIDE

Codebase Architecture: What Each Group of Files Does

1. Compiler Frontend Core (compiler.cpp)

- Token definitions: 50+ token types for environment, shapes, 3D, and games.
- Lexer & Scanner: Regex/string analysis, stripping comments (#), token stream.
- Syntax Analyzer & AST: Parameter count checking and syntax enforcement.
- Code Generator: Emits generated_game.c and triggers GCC build command.

2. Runtime Backend Engine (graphics_runtime.h)

- Hardware Lifecycle: SDL2 window creation & OpenGL 2.1 context management.
- Shader Engine: Compiles GLSL fragment shaders (Plasma, Grid, CRT, Starfield).
- 2D & 3D Math: Trigonometric circle approximations, matrix stacks, 3D camera.
- Physics & Collision: Pong ball reflection physics and 3D wall sliding collision.

3. DSL Game & Graphics Programs (*.my)

- demo.my: Original Review 1 baseline proof-of-concept.
- showcase.my: 10+ 2D shapes, 35+ named colors, and linear gradient fills.
- cyberpunk.my: GLSL fragment shaders, gradients, and affine transformations.
- pong.my: Full interactive 2D Pong game with AI paddle and live scoreboard.
- world3d.my: Navigable 3D first-person POV room with collision obstacles.

4. Intermediates & Native Artifacts

- generated_game.c: Intermediate ANSI C code generated by compiler.cpp.
- mycompiler: The compiled C++ compiler binary (built via g++ compiler.cpp).
- final_game: The final linked OpenGL 2.1 + SDL2 desktop graphics executable.
- plasma.frag: User-loadable custom GLSL fragment shader source file.

COMPILER FRONTEND

Frontend Compiler: Token Enum, Lexer & Abstract Syntax Tree

compiler.cpp — Token Definitions & AST Node Structure

Frontend Compiler: Token Enum & AST Architecture (compiler.cpp)

```cpp
enum TokenType {
    TOKEN_WINDOW, TOKEN_TITLE, TOKEN_BG, TOKEN_SHADER,
    TOKEN_PONG, TOKEN_PADDLE, TOKEN_AI_PADDLE, TOKEN_BALL,
    TOKEN_3D, TOKEN_FLOOR, TOKEN_WALLS, TOKEN_CUBE, TOKEN_PILLAR,
    TOKEN_ROTATE, TOKEN_TRANSLATE, TOKEN_SCALE,
    TOKEN_RECTANGLE, TOKEN_CIRCLE, TOKEN_IDENTIFIER, TOKEN_NUMBER
};

struct Token {
    TokenType type;
    string value;
    int line;
};

// Abstract Syntax Tree (AST) Base Node
struct ASTNode {
    virtual ~ASTNode() = default;
    virtual void codegen(ofstream &out) = 0;
};
```

Lexer & Parser Mechanics

- **Token Classification:**
  Categorizes source text into Window, Shapes, Gradients, Transformations, 3D Commands, and Game Entities.
- **Smart Color Tokenizer:**
  The lexer intercepts named colors (gold, cyan, lime), hex strings (#RRGGBB), and numeric triplets (255 128 0), unifying them into ColorRGB structs.
- **Comment & Whitespace Handling:**
  Strips shell-style comments (# ...) and normalizes tabs/spaces while preserving accurate line tracking for syntax error messages.
- **AST Node Hierarchy:**
  Each statement maps to an AST node that enforces argument count and type requirements before code emission begins.

CODE GENERATION

Backend Transpiler: Code Generation & Automated GCC Linker

From DSL Tokens to High-Speed C Code

Intermediate Code Emission

The compiler outputs generated_game.c, transforming high-level DSL primitives into clean C runtime function calls.

Display List Pre-compilation

Static geometry is wrapped inside glNewList(scene, GL_COMPILE) blocks, offloading primitive vertex calculations to the GPU driver during setup.

Mode Flag Detection

Detects is_pong_mode and is_3d_mode flags to route control flow to run_pong_game() or run_3d_engine() respectively.

Automated Native Linker

Runs: gcc generated_game.c -lSDL2 -lGL -lGLU -lm -o final_game seamlessly in the background without needing a manual Makefile.

compiler.cpp — Code Generation Logic

Transpiler Code Generation: AST to C Runtime Calls (compiler.cpp)

```
else if (t == TOKEN_CUBE) {
    string x = tokens[i+1].value;
    string y = tokens[i+2].value;
    string z = tokens[i+3].value;
    string size = tokens[i+4].value;
    size_t next_idx = i + 5;
    ColorRGB c = parse_color_token(tokens, next_idx);
    out_file << "    add_3d_cube(" << x << ", " << y << ", " << z
             << ", " << size << ", "
             << c.r << ", " << c.g << ", " << c.b << ");" << endl;
    i = next_idx - 1;
}
else if (t == TOKEN_PONG) {
    is_pong_mode = true;
}
```

GRAPHICS ENGINE

Runtime Backend: OpenGL 2.1 & GLSL Fragment Shader Pipeline

graphics_runtime.h — GLSL Shader Compiler & Linker

Backend Engine: OpenGL 2 GLSL Shader Pipeline (graphics_runtime.h)

```
1  static GLuint compile_shader_source(const char* frag_src) {
2      GLuint v_shader = glCreateShader(GL_VERTEX_SHADER);
3      glShaderSource(v_shader, 1, &default_vshader_src, NULL);
4      glCompileShader(v_shader);
5
6      GLuint f_shader = glCreateShader(GL_FRAGMENT_SHADER);
7      glShaderSource(f_shader, 1, &frag_src, NULL);
8      glCompileShader(f_shader);
9
10     GLuint prog = glCreateProgram();
11     glAttachShader(prog, v_shader);
12     glAttachShader(prog, f_shader);
13     glLinkProgram(prog);
14     return prog;
15 }
```

Hardware Engine Features

Fixed & Programmable Dual Pipeline

Combines high-speed OpenGL 2 fixed-function display lists with programmable GLSL fragment shaders.

Dynamic Uniform Passing

Automatically passes u_time (seconds elapsed since launch) and u_resolution (width, height) to active fragment shaders at 60 FPS.

8+ Built-in Shaders

Plasma, Cyberpunk Grid, Waves, CRT Scanlines, Starfield, Neon Tunnel, Water Ripple, and Custom Shader file loading.

Live Hot-Swapping [SPACE]

Users can press the Spacebar during runtime to cycle between GPU shaders on the fly without restarting the application.

MATH & PHYSICS

Physics & Mathematics: 2D & 3D Real-Time Collision Engines

graphics_runtime.h — 3D Camera & Wall Sliding Collision

Runtime Engine: 3D Camera & Collision Math (graphics_runtime.h)

```
// 3D Player Movement with Smooth Wall-Sliding Collision
float nx = cam_pos_x + move_x;
float nz = cam_pos_z + move_z;

// Check collision with 3D perimeter walls and obstacle cubes
if (!check_3d_cube_collision(nx, cam_pos_y, cam_pos_z, 0.4f)) {
    cam_pos_x = nx; // Allow movement along X axis
}
if (!check_3d_cube_collision(cam_pos_x, cam_pos_y, nz, 0.4f)) {
    cam_pos_z = nz; // Allow movement along Z axis
}

// Update first-person camera look vectors
gluLookAt(cam_pos_x, cam_pos_y, cam_pos_z,
          cam_pos_x + target_x, cam_pos_y + target_y, cam_pos_z + target_z,
          0.0f, 1.0f, 0.0f);
```

Collision Formulas & Implementation

2D Pong: AABB vs. Circle

Collision occurs when the ball center is within paddle bounds expanded by the ball radius:

$|x_{ball} - x_{paddle_center}| \le w/2 + r$ AND

$|y_{ball} - y_{paddle_center}| \le h/2 + r$.

Angles dynamically deflect based on paddle impact offset.

3D Wall-Sliding Collision

Candidate positions (nx, nz) are tested independently against cube/wall boundaries. If an obstacle blocks movement along X, nx is clamped while nz remains free — producing realistic wall-sliding motion.

AI Paddle Reaction Modeling

AI tracks the ball's Y-coordinate with intentional reaction delay and bounded speed, creating a balanced, competitive opponent.

OUTPUT DEMONSTRATION

Visual Output 1: 2D Geometry & 35+ Named Color System (showcase.my)

2D Vector Primitives & Palette

Rich Geometry Suite

Supports rectangles, rounded rectangles, squares, centered boxes, circles, and ellipses in both filled and outline modes.

35+ Color Dictionary

Built-in named colors: cyan, magenta, gold, coral, emerald, indigo, teal, violet, neon_green, crimson.

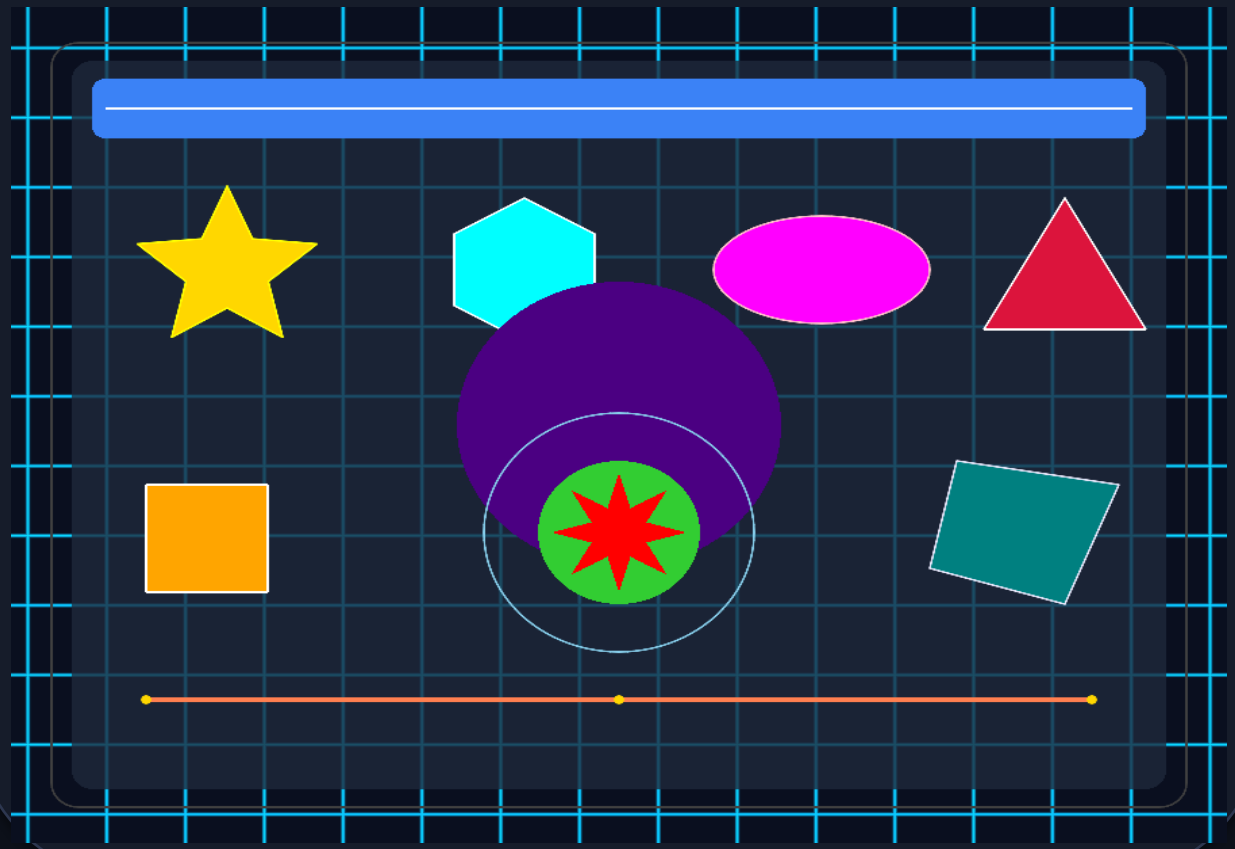
Hex Color Support

Direct support for standard web hex colors like #ff5722, #00d2ff, #1e1e2e.

Execution Command

./mycompiler showcase.my && ./final_game

Actual Running Output: 2D Shapes & Anti-Aliased Vector Geometry



OUTPUT DEMONSTRATION

Visual Output 2: GPU Fragment Shaders & Transformations (cyberpunk.my)

GPU Shaders & Transformations

Live Fragment Shader Background

Executes real-time animated GLSL shader code calculating per-pixel plasma and grid effects at 60 FPS.

Multi-Color Linear Gradients

gradient_rect creates smooth color transitions between vertical and horizontal stops.

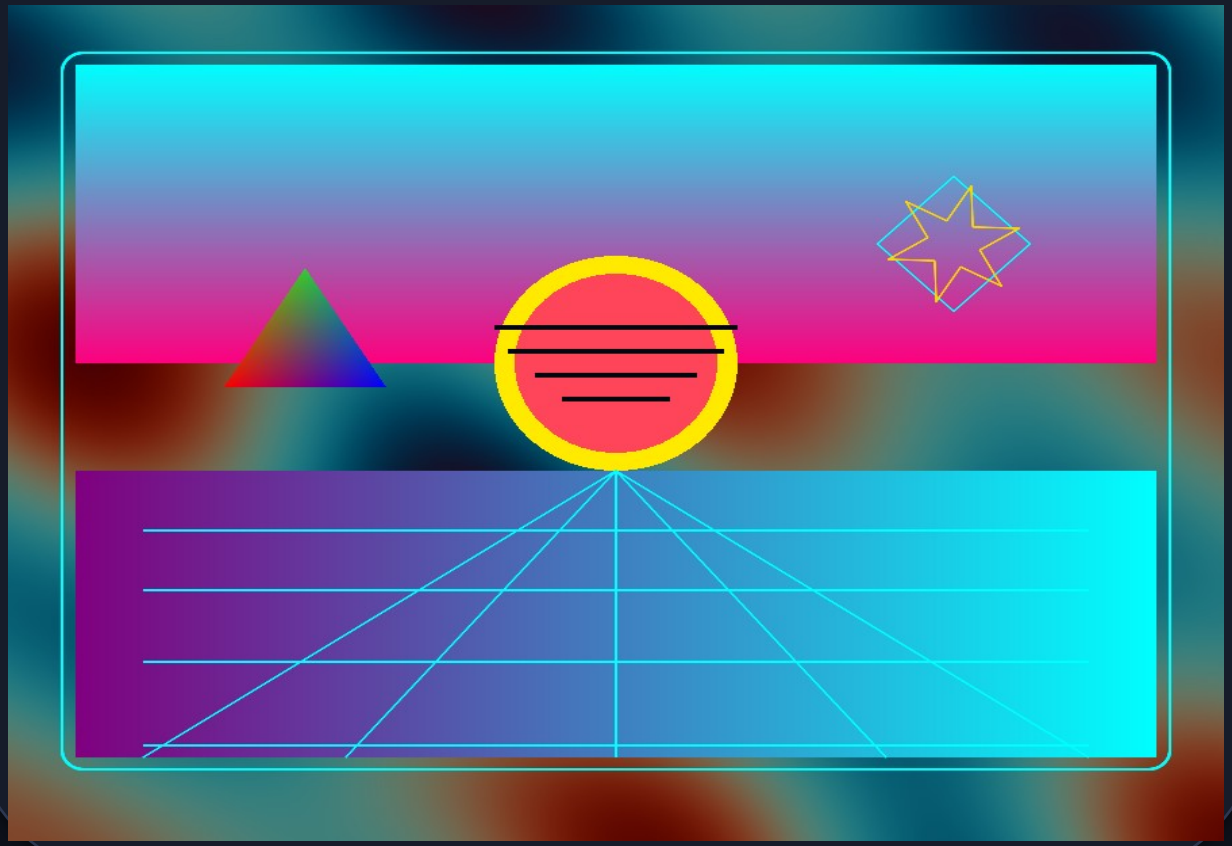
Matrix Stack Transformations

rotate <deg>, translate <dx> <dy>, scale <sx> <sy>, with push and pop scoping.

Execution Command

./mycompiler cyberpunk.my && ./final_game

Actual Running Output: Animated GPU Fragment Shader & Cyberpunk Aesthetics



GAME ENGINE DEMO

Visual Output 3: Interactive 2D Pong Game Engine (pong.my)

DSL Game Script: pong.my (Only 10 Lines!)

pong.my — 2D Interactive Game Script

```
# Cyberpunk Pong - Interactive Player vs AI Game
title "Cyberpunk Pong - Player vs AI (OpenGL 2)"
window 800 600
bg #060614

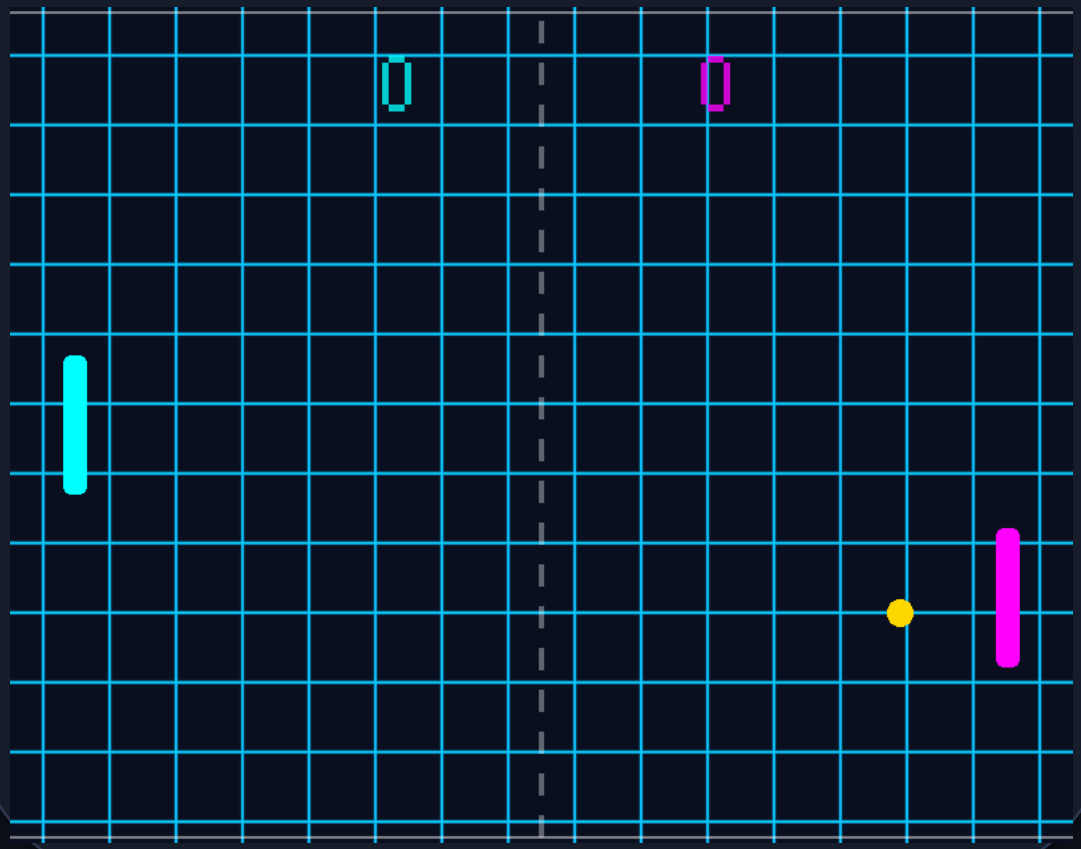
# Live Hardware GLSL Background Shader
shader grid

# Activate Pong Engine Mode & Customize Entities
pong
paddle cyan
ai_paddle magenta
ball gold
```

Interactive Features in pong.my

- Player 1 controls paddle via W / S or Up / Down.
- Autonomous AI tracks ball position in real-time.
- Live retro bitmap font scoreboard and reset key (R).
- Press [SPACE] live to switch background shaders!

Actual Running Output: Hardware-Accelerated 2D Pong Game



3D ENGINE DEMO

Visual Output 4: 3D First-Person POV World (world3d.my)

DSL 3D Script: world3d.my

world3d.my — 3D First-Person POV Environment Script

```
# 3D First-Person World Explorer
title "3D First-Person POV World (OpenGL 2 Engine)"
window 900 700
bg #060614

mode_3d
floor 40 #161a2e #0e1224
walls 40 5 #0f3460

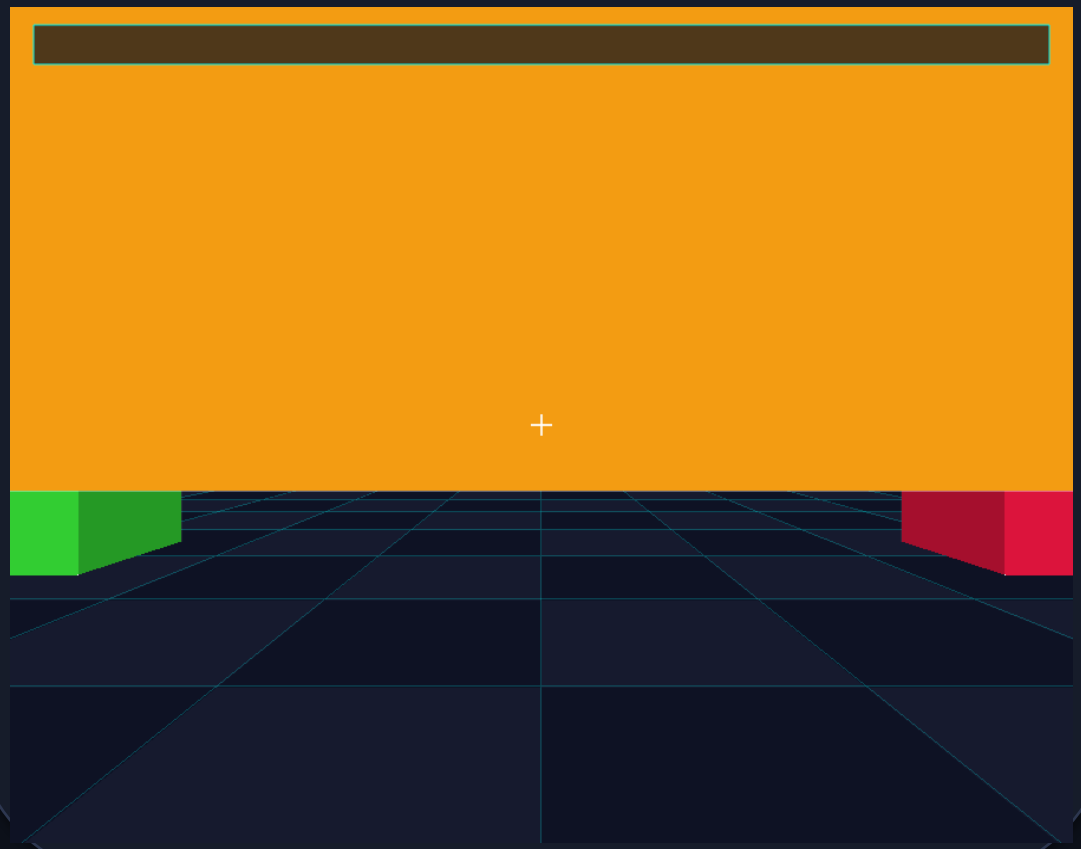
# Cubes with Built-in Collision Detection
cube 0 1 -10 2 gold
cube -8 1 -6 2 cyan
cube 8 1 -6 2 magenta
cube -6 1 6 2 lime
cube 6 1 6 2 crimson

# Monolith Pillars
pillar -12 0 2 5 #e94560
pillar 12 0 2 5 #00adb5
pillar 0 12 2 5 #f39c12
```

3D Engine Features in world3d.my

- mode_3d: Enables OpenGL depth-buffer (GL_DEPTH_TEST).
- First-person POV camera with WASD and mouse-look.
- Checkerboard floor grid, perimeter walls, and pillars.
- Obstacle collision detection prevents walking through cubes.

Actual Running Output: Real-Time Navigable 3D Environment



MILESTONE COMPARISON

Evolution: Review 1 Mock Prototype vs. Current Hardware Engine

Architectural Growth Across Project Milestones

Windowing & Rendering

Review 1: Mock printf() statements (mock_runtime.h) simulating drawing.

➔Current: True OpenGL 2.1 + SDL2 GPU rendering at native 60 FPS double-buffered.

GPU Shaders

Review 1: None. Purely CPU-side text placeholders.

➔Current: 8+ dynamic GLSL fragment shaders (Plasma, Grid, CRT) + custom shader loading.

Interactive Games

Review 1: Static geometry output only.

➔Current: Full interactive 2D Pong game with ball physics, AI tracking paddle, and score display.

3D Graphics Support

Review 1: 2D primitives only.

➔Current: Complete 3D first-person POV environment with depth buffer, camera look, and collision.

Math & Physics

Review 1: Basic integer coordinates.

➔Current: Floating-point circle-AABB collision, dynamic reflection vectors, and 3D wall sliding.

ROADMAP & CONCLUSION

Future Implementations & Technical Roadmap

Technical Roadmap & Next Steps

3D Lighting & Shading Pipeline

Integrate Phong and Blinn-Phong lighting models, supporting directional sunlight, ambient reflection, and point lights.

3D Model / Asset Loader (.OBJ / .GLTF)

Allow DSL scripts to import external 3D meshes: 'load_mesh "model.obj" 0 0 0'.

Sound & Audio Engine (SDL_mixer)

Integrate background music and synthesized 8-bit sound effects for collisions, scores, and ambient atmospheres.

Rigid-Body Physics Engine

Expand collision detection with mass, gravity, friction, and restitution coefficients for complex interactions.

Turing-Complete DSL Extensions

Introduce user-defined variables, loop constructs (repeat 10 { ... }), and conditional branching.

Project Summary & Deliverables

Transpiler Pipeline

Successfully proves that a custom domain-specific language can compile to native OpenGL 2 + SDL2 without performance loss.

Milestone Completion

All core goals achieved: 2D primitives, 35+ colors, GPU shaders, Pong AI game, and 3D first-person world.

Academic Documentation

Accompanied by a 14-page formal project report (.docx) and full technical documentation (README.md).

Open-Source Verification

Publicly hosted and verified online at: github.com/eldhoGI2/shader-compiler.

Thank You! Open for Questions & Live Demonstration.